

Índice

Descripción general	2
Arquitectura del Agentic RAG	3
1. Carga y preprocesamiento de datos	3
2. Chunking y embeddings	5
3. Recuperación semántica especializada mediante herramientas (tools)	8
4. Razonamiento agentic y generación controlada	11
Prompt Engineering	16
Estrategias de Prompt Engineering Aplicadas	17
Desafíos encontrados y soluciones implementadas	19
1. Mezcla de idiomas en la generación	19
2. Problemas con el AgentState y la memoria interna	19
3. Nodo separado para small talk	19
4. Top-k demasiado bajo (Top 3) en la búsqueda semántica del dataset de películas	20
5. "Consultar memoria" como feature interna (sin tool-calling)	20
Pruebas y resultados obtenidos	21
Evidencia	21
1. Saludo Conversacional (Small-talk)	21
2. Memoria a Corto Plazo (Ingreso de Dato)	22
3. Recuperación Simple (RAG sobre Dataset)	22
4. Síntesis de Documentos Largos (RAG sobre Papers)	23
5. Búsqueda por Atributo (Metadatos)	23
6. Combinada: RAG + Reasoning + Memoria	24
7. Consulta Multi-Criterio (Filtros Implícitos)	25
8. Memoria Conversacional Inmediata (Historial)	28
9. Cita de Fuente (Anti-alucinación)	30
10. Cierre Conversacional	31
Interfaz con Gradio	32
Investigación: posibles mejoras	33
Fuentes consultadas	34

Descripción general

Se desarrolló un sistema Agentic RAG capaz de responder preguntas sobre un dataset de películas y sobre un conjunto de papers académicos, integrando recuperación de información, razonamiento y toma de decisiones.

El sistema se compone de tres módulos principales:

- RAG Pipeline para dos fuentes de conocimiento:
 - Dataset estructurado de películas.
 - Conjunto de documentos PDFs (papers académicos).
- Agente con tool-calling implementado con *LangGraph*.
El agente decide dinámicamente cuándo:
 - Consultar el dataset de películas.
 - Buscar conceptos teóricos en los PDFs.
 - Acceder a la memoria reciente.
 - Responder directamente en casos de *small-talk*.
- Memoria a corto plazo integrada al estado del agente, que almacena las últimas interacciones para permitir referencias como “eso”, “lo anterior”, “qué te dije”, y habilitando un flujo conversacional coherente.

La arquitectura resultante permite construir un chatbot capaz de mantener contexto, evitar alucinaciones, combinar razonamiento con recuperación de información y utilizar herramientas solo cuando es necesario.

Arquitectura del Agentic RAG

La arquitectura implementada corresponde a un Agentic RAG cuyo objetivo es integrar datos heterogéneos (dataset estructurado y documentos extensos), herramientas externas, recuperación semántica y razonamiento controlado.

El sistema está diseñado para separar explícitamente recuperación y generación, minimizar alucinaciones y garantizar que el modelo utilice evidencia verificable antes de producir contenido.

El pipeline se estructura en cuatro capas:

1. Carga y preprocesamiento de datos
2. Chunking y embeddings
3. Recuperación semántica especializada mediante herramientas (*tools*)
4. Razonamiento agentic y generación controlada

Las dos primeras capas preparan los insumos. La arquitectura del Agentic RAG se concentra principalmente en las capas 3 y 4, donde ocurre la combinación entre decisión, recuperación y síntesis.

1. Carga y preprocesamiento de datos

En esta primera capa se integraron y normalizaron las dos fuentes de conocimiento utilizadas en el sistema:

- Un dataset estructurado de películas (*Letterboxd*)¹
- Seis artículos académicos provistos para el Obligatorio²

El objetivo principal fue obtener textos homogéneos, limpios y consistentes, de manera de maximizar la calidad de los embeddings y garantizar una recuperación semántica eficaz en etapas posteriores.

Carga y análisis del dataset de películas

El dataset seleccionado contiene 28 columnas con información detallada sobre cada película. El archivo CSV se cargó en un DataFrame con validación explícita de errores (FileNotFoundError, ParserError) para asegurar robustez ante formatos inválidos o rutas incorrectas.

¹ obtenido de Kaggle, disponible en [Letterboxd Movies Dataset - 16K+ Films with Compre](#)

²  Papers Obligatorio 2025

Posteriormente, se realizó un análisis inicial para identificar valores nulos y determinar las estrategias de normalización requeridas.

Limpieza y estandarización de campos

Los valores faltantes se completaron con expresiones explícitas como "Unknown", "Genre not specified" o 0 en el caso de la duración (runtime).

Este enfoque evita que los embeddings incorporen ruido derivado de valores NaN, y a la vez preserva información útil para búsquedas posteriores.

Construcción del campo unificado text_content

Se decidió incorporar una nueva columna en la que se transformó cada película en un bloque de texto coherente que consolida toda la información de la película: título, país, año y década, duración, idioma, género principal, escala de producción, y flags de “clásico” o “reciente”.

Esta decisión se basa en que unificar los atributos en un único texto mejora la densidad semántica del embedding (al consolidar múltiples atributos en un único texto, el modelo captura relaciones semánticas que se perderían si los campos permanecieran aislados), evita fragmentación de información entre columnas y garantiza que consultas complejas (por ejemplo “películas recientes de terror de USA en los 2000s”) puedan resolverse por similitud semántica.

Langchain Documents

Cada fila del data frame se convierte en un Document de LangChain con:

- page_content = text_content
- metadata = {country, decade, language, primary_genre, ...}

El uso de Document fue necesario porque LangChain y Pinecone operan nativamente sobre objetos Document y además permite búsquedas híbridas: similitud semántica (por embeddings) o filtrado estricto por metadata (país, década, idioma).

Carga de documentos PDF

La segunda fuente de datos corresponde a los seis papers provistos para el Obligatorio. Cada PDF se carga mediante PyPDFLoader, generando un Langchain Document por página. La ventaja de generar un Document por página es que no solo se mantiene

coherencia temática sin mezclar secciones heterogéneas, sino que también los embeddings funcionan mejor con textos moderados (~300–700 tokens).

Preprocesamiento del contenido de los papers

Los textos de papers técnicos contienen ruidos frecuentes: acrónimos, inconsistencias, stopwords, repeticiones, saltos de línea y símbolos no convencionales. Por tanto, se consideró crucial preprocesar los textos para mejorar la calidad de los embeddings.

Según las mejores prácticas, se aplicaron las funciones diseñadas en los laboratorios del curso:

- Lematización: Reducir palabras a su forma raíz
- Expansión de abreviaturas: Convertir abreviaciones a formas completas
- Eliminación de stopwords: Remover palabras sin significado semántico
- Normalización: Limpiar caracteres especiales y espacios

Estas técnicas mejoran la capacidad de coincidencia semántica y reducen el ruido en los embeddings. El texto resultante es homogéneo y más útil para búsquedas vectoriales.

2. Chunking y embeddings

Tras el preprocesamiento, la segunda capa del pipeline consistió en aplicar estrategias de chunking y generación de embeddings optimizadas para cada dominio. Esta etapa es esencial para garantizar que la recuperación semántica sea efectiva y que los documentos indexados mantengan suficiente coherencia contextual para responder preguntas complejas.

Para la etapa de indexación se utilizó Pinecone como vector database y la abstracción VectorStore de LangChain, debido a que permiten realizar búsquedas de similitud eficientes sobre millones de embeddings, soportar metadata estructurada y escalar sin necesidad de gestionar infraestructura propia. Estas capacidades son esenciales para un sistema RAG con múltiples dominios semánticos.

Análisis de longitud para definir estrategia de chunking en el dataset de películas

Como primer paso, se analizó la distribución de longitud del texto consolidado (text_content) en el dataset de películas. Se obtuvo como media 293 caracteres, siendo el mínimo y máximo 256 y 392 respectivamente.

Dado que se trata de textos cortos y coherentes, se concluyó que no era necesario fragmentarlos, por lo que cada película se conservó como una única unidad semántica.

Si bien es recomendable usar chunks pequeños solo para textos largos (>500–800 caracteres), en este caso, cada película está por debajo de ese umbral y constituye un concepto atómico, por lo que dividirla hubiese degradado la coherencia del embedding.

Chunking del dataset de películas

Se mantuvo un proceso formal de chunking utilizando *RecursiveCharacterTextSplitter* para preservar homogeneidad entre dominios. Se usó un chunk size de 500 y un chunk overlap de 50.

La división mantuvo intacta los textos, generando un chunk por película. Adicionalmente, se garantizó que la metadata clave se transfiriera de forma completa a cada chunk, para habilitar búsquedas híbridas posteriores.

Chunking de los documentos PDF

A diferencia del dataset de películas, los artículos académicos contienen textos extensos y densos, con conceptos que requieren mayor contexto.

Por ello, se utilizó una estrategia diferente, seleccionando un chunk size de 700 y un chunk overlap de 100. El uso de chunks medianos se justifica dado que son documentos técnicos largos y no se desea cortar definiciones importantes. Además, el overlap de 100 permite mantener continuidad conceptual en párrafos densos.

Esta configuración evita que conceptos clave queden fragmentados y pierdan significado durante la indexación.

Modelo de embeddings

Para ambos dominios se empleó un único modelo: *sentence-transformers/all-MiniLM-L6-v2*, óptimo para RAG livianos.

El uso de un único modelo de embeddings es fundamental para mantener una base semántica coherente. Mezclar modelos distintos podría introducir incompatibilidades vectoriales y degradar la similitud.

Además, este modelo fue el recomendado en el curso para implementaciones de RAG con Pinecone por su balance entre velocidad y precisión.

Creación y configuración de índices en Pinecone

Se utilizaron dos índices independientes:

- Índice 1: letterboxd
 - Chunks del dataset de películas
 - Dimensión: 384
 - Métrica: cosine
- Índice 2: rag-pdfs-obligatorio
 - Chunks derivados de los PDFs
 - Dimensión: 384
 - Métrica: cosine

Las semánticas de ambos dominios son radicalmente distintas, mezclarlas en un mismo índice habría producido interferencias semánticas. Separar los índices permite mejorar precisión, usar filtros específicos por metadata y evitar que consultas teóricas devuelvan películas o viceversa.

Indexación de documentos

Finalmente, se indexaron todos los chunks en Pinecone utilizando *PineconeVectorStore.from_documents*.

Este proceso genera automáticamente IDs únicos para cada chunk, almacenamiento del vector, indexación completa de la metadata asociada y compatibilidad inmediata con herramientas de recuperación semántica.

Decisión de diseño: por qué Pinecone y VectorStore

La elección de Pinecone como vector database se fundamenta en:

- Búsqueda Vectorial Optimizada: Pinecone está diseñado específicamente para recuperación por similitud con métricas como cosine, dot-product o euclidean, con latencias muy bajas y estructuras altamente optimizadas.
- Escalabilidad sin gestionar infraestructura: A diferencia de alternativas locales (FAISS, Chroma en CPU), Pinecone permite manejar grandes volúmenes sin preocuparse por memoria, concurrencia o persistencia.
- Metadata Filtering nativo: Pinecone permite combinar filtros estructurados por metadata (país, género, década, idioma) junto con similitud semántica. En este caso, esto fue crítico para consultas complejas como “Películas de terror de los 90s producidas en Francia”.

Por su parte, LangChain abstrae el acceso a bases vectoriales mediante el modelo VectorStore, lo que permite:

- Reutilizar otra base en el futuro sin modificar el código del pipeline.
- Acceder a funciones unificadas para añadir documentos, realizar similarity search, obtener retrievers.
- Manejar metadata asociada a cada chunk de forma nativa.

Esta combinación (Pinecone + VectorStore) proporciona una base semántica robusta, escalable y con capacidades híbridas (vector + metadata) necesarias para un RAG agentic.

3. Recuperación semántica especializada mediante herramientas (*tools*)

En esta tercera capa se implementa el mecanismo de recuperación semántica que utiliza el agente para obtener evidencia antes de generar una respuesta.

La recuperación no se ejecuta automáticamente: el agente sólo la realiza cuando detecta que la consulta del usuario requiere información del dataset de películas, del conjunto de papers o de la memoria de corto plazo.

Para lograr esto, se desarrollaron tres herramientas (*tools*) que encapsulan la lógica de búsqueda contextual, normalización de consultas y filtrado por metadata.

La separación explícita entre herramientas permite mantener un flujo de recuperación modular y altamente especializado para cada dominio.

Objetivo del diseño de las tools

Cada tool cumple cuatro funciones esenciales:

- Normalizar la consulta del usuario, convirtiendo términos coloquiales o ambiguos a un formato estándar compatible con los índices vectoriales.
- Construir filtros por metadata (país, década, idioma, género, duración) cuando corresponda.
- Realizar recuperación semántica real desde Pinecone, evitando depender de contenido inventado.
- No responder si no se encuentra evidencia, garantizando que la generación posterior esté basada únicamente en documentación existente.

Este diseño sigue la regla central del pipeline: el agente no puede generar contenido factual sin evidencia recuperada previamente.

Tool: buscar_peliculas

Esta herramienta se utiliza para todas las consultas relacionadas con el dataset de *Letterboxd*.

Para normalizar la consulta, como primer paso se aplican reemplazos para convertir expresiones del lenguaje natural a los valores estandarizados del dataset, por ejemplo:

- “EEUU”, “Estados Unidos”, “United States” → USA
- “terror”, “miedo” → horror
- “romántica” → romance
- “años 2000”, “2000s” → 2000s decade

Esto es crucial, ya que el dataset usa etiquetas estandarizadas y sin esta normalización bajarían significativamente los *recall scores*.

Luego, también fue posible generar filtros dinámicos por metadata. Si la consulta menciona países, décadas, géneros, idioma o duración, se construye automáticamente un filtro del estilo:

```
{  
    "country": "USA",  
    "decade": 2000,  
    "primary_genre": "Horror"  
}
```

Esto habilita búsquedas híbridas en Pinecone, utilizando similitud semántica y coincidencias exactas por metadata, aumentando precisión.

Finalmente se realiza la búsqueda vectorial, en este caso utilizando `similarity_search` con un `k=10`: `vectorstore_letterboxd.similarity_search(query, k=10, filter=filtro)`. Se decidió obtener los top-10 resultados más relevantes para aumentar la probabilidad de recuperar algo útil dada la gran cantidad de películas en el dataset.

Si no hay resultados relevantes, la herramienta responde explícitamente que no encontró evidencia, evitando respuestas incorrectas.

Tool: buscar_papers

Esta herramienta se utiliza para consultas teóricas sobre los papers.

Para normalizar la consulta, dado que estos documentos contienen terminología densa, se usa una tabla de expansión adaptada al dominio:

- “rag” → retrieval augmented generation
- “embeddings” → vector representations embeddings
- “attention” → self-attention mechanism
- “gpt” → generative pretrained transformer

Esto mejora el alineamiento entre la consulta del usuario y el contenido real de los papers.

Luego se realiza la búsqueda vectorial, nuevamente utilizando `similarity_search`, pero con un `k=3`: `vectorstore_pdfs.similarity_search(query, k=3)`. Se decidió obtener los top-3 resultados más relevantes para no recuperar fragmentos redundantes o ruido técnico.

Como en el caso anterior, si no hay coincidencias, la tool responde explícitamente que no existe evidencia.

Tool: consultar_memoria

La memoria de corto plazo almacena las últimas 5 interacciones relevantes del usuario. Esta herramienta se activa cuando el usuario utiliza referencias como “¿qué te dije antes?”, “repetí lo último”, “eso que mencionaste”, “como te dije recién”.

La tool implementa detección de referencias explícitas (“última respuesta”, “lo anterior”, etc.), búsqueda por palabras clave dentro del historial reciente y coincidencias parciales para casos ambiguos.

Si no encuentra información asociada, responde explícitamente que la memoria no contiene datos relevantes.

Es importante destacar que la memoria nunca inventa información, ni se usa como fuente factual. Su propósito es únicamente mantener continuidad conversacional.

Justificación técnica del uso de *tools*

El uso de *tools* separadas responde a varios objetivos:

- Evitar alucinaciones: el agente no puede producir contenido factual sin antes recuperar evidencia real.
- Modularidad: cada dominio semántico (películas, papers, memoria) se implementa como una herramienta independiente.
- Escalabilidad: permite agregar nuevas herramientas sin modificar la lógica central del agente.
- Rendimiento: la normalización previa mejora la efectividad de las búsquedas y reduce consultas fallidas.

- Transparencia y auditabilidad: cada recuperación queda claramente identificada dentro del flujo del agente.

4. Razonamiento agentic y generación controlada

El agente se diseñó siguiendo un enfoque agentic, donde cada paso del razonamiento está explícitamente representado mediante un grafo de ejecución. En lugar de delegar todo el control de la interacción al LLM, la arquitectura utiliza LangGraph para imponer un flujo determinista y verificable entre razonamiento, recuperación y generación.

Este enfoque permite controlar cuándo se recupera evidencia, cuándo se puede generar contenido, y garantizar que la respuesta final esté siempre fundamentada.

El funcionamiento del agente se organiza alrededor de tres componentes principales:

- Un estado conversacional estructurado (AgentState): Mantiene el historial de mensajes y una memoria de corto plazo. Al estar formalizado como un estado del grafo, cada nodo puede leerlo y actualizarlo de forma consistente.
- Un nodo de decisión (chatbot_node): Evalúa el mensaje del usuario, detecta intención (small talk / referencia al historial / consulta sobre películas / consulta teórica) y decide si debe activar una herramienta.
- Un conjunto de herramientas especializadas (consultar_memoria, buscar_peliculas, buscar_papers): Son invocadas únicamente cuando el agente determina que debe recuperar evidencia factual antes de generar la respuesta final.

Estado conversacional y memoria de corto plazo

El agente define un estado especializado heredando de MessagesState. Esta estructura mantiene:

- Historial completo de mensajes (rol + contenido), administrado por LangGraph.
- Una memoria de corto plazo (memory), implementada como una lista acotada a las últimas cinco interacciones relevantes.

La memoria se actualiza mediante la función guardar_memoria, que agrega cada intercambio {mensaje, respuesta} al estado y también a una memoria global auxiliar. Esta capa no almacena conocimiento factual; su propósito es estrictamente conversacional, permitiendo resolver referencias como “¿qué te dije antes?” o “eso que mencionaste”.

Este diseño garantiza continuidad en el diálogo sin contaminar la recuperación factual, que siempre proviene de los datasets vectorizados.

A diferencia de los índices vectoriales (Pinecone), la memoria no se implementa como un vectorstore porque almacena información efímera que cambia constantemente, por lo que indexarla generaría sobrecarga innecesaria. Además no requiere búsquedas por similitud semántica, sino detecciones textuales simples y referencias explícitas.

Nodo principal de decisión: detección de intención

El nodo central del grafo, `chatbot_node`, implementa la lógica de razonamiento inicial del agente. Su función es interpretar el mensaje recibido, decidir si corresponde activar el mecanismo de recuperación y determinar el siguiente nodo en el flujo de ejecución.

Uso de LLM con herramientas

El LLM utilizado se encuentra vinculado explícitamente a las herramientas desarrolladas mediante `llm.bind_tools(tools)`

Esto es fundamental para el diseño agentic:

- El LLM no solo genera texto: puede emitir tool-calls.
- La elección de la herramienta adecuada depende del razonamiento interno del LLM, no de reglas externas.
- El agente no necesita clasificadores ni if/else basados en keywords.
- El modelo puede incluso invocar más de una herramienta en una misma consulta, si lo considera necesario para obtener evidencia suficiente.

En este sentido, el LLM funciona como un clasificador semántico de intención, guiado por las descripciones de cada tool dentro del prompt de sistema.

Jerarquía de decisión: única detección manual - *small talk*

Aunque el LLM podría decidir todo, se implementó una optimización, la única detección de intención completamente manual es la identificación de *small talk*. Esto evita llamadas innecesarias a herramientas para mensajes triviales, mejora fluidez y reduce costo computacional.

Si el mensaje coincide con saludos, agradecimientos o frases sociales breves, `chatbot_node` responde directamente con el LLM sin activar herramientas.

Decisiones delegadas al LLM

Después de descartar *small talk*, todo el resto de la detección de intención queda en manos del LLM gracias a la integración con tools.

El flujo es el siguiente:

- El mensaje se envía al LLM con una prompt que define qué hace cada herramienta, cuándo debe utilizarla y la regla estricta de no generar contenido factual sin evidencia.
- El LLM decide automáticamente si necesita:
 - invocar consultar_memoria
 - invocar buscar_peliculas
 - invocar buscar_papers
 - responder directamente
- Si necesita evidencia, el LLM genera un tool_call, activando la capa de recuperación.

Esto significa que el agente no implementa reglas manuales para detectar consultas sobre películas, teoría o memoria; estas decisiones emergen del razonamiento del modelo al evaluar la consulta y comparar su contenido con la descripción funcional de cada herramienta.

Activación del nodo de *tools*

Cuando el LLM decide que debe ejecutar una *tool*, produce una tool_call. LangGraph detecta esta llamada mediante tools_condition y desvía el flujo al nodo: chatbot_node → tools → chatbot_node

El nodo de *tools* ejecuta la herramienta correspondiente, devuelve los resultados y transfiere el control nuevamente al nodo principal. A partir de esa evidencia, chatbot_node produce la respuesta final con el LLM.

Si no hubo tool-calls, el flujo termina directamente en la generación.

Representación explícita del flujo mediante un grafo

El uso de LangGraph permite representar el razonamiento del agente como un grafo explícito, garantizando:

- Transparencia: cada decisión es inspeccionable.
- Control determinista: evita generar sin evidencia.
- Modularidad: nuevas tools pueden agregarse sin modificar lógica interna.
- No recurrencia: se evita que el agente “loopee” o vuelva a ejecutar tools de forma incorrecta.

El grafo final se estructura de la siguiente forma:

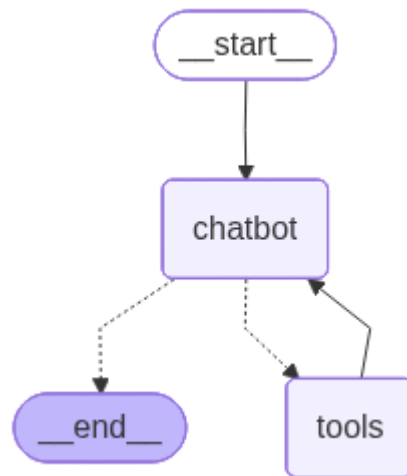


Ilustración 1

Configuración del modelo generativo (LLM)

La etapa final del razonamiento agentic consiste en la generación de la respuesta, y para ello se configuró un modelo LLM optimizado para operar dentro del grafo de LangGraph y en conjunto con herramientas especializadas.

Para el agente se seleccionó el modelo Qwen/Qwen3-4B-Instruct, ejecutado mediante HuggingFaceEndpoint.

La elección se fundamenta en tres criterios principales:

- **Tamaño moderado (4B):** Un agente RAG agentic no requiere que el LLM “sepa” el contenido factual; la evidencia viene de Pinecone. Por eso, un modelo compacto es suficiente y además reduce latencia, costos y evita respuestas excesivamente verbosas.
- **Capacidad de tool-calling y razonamiento estructurado:** La variante Instruct está optimizada para seguir instrucciones, manejar descripciones de herramientas y producir tool_calls consistentes, lo cual es esencial para el grafo de ejecución.
- **Equilibrio entre precisión y eficiencia:** Qwen3-4B se ubica en un punto óptimo dentro de los modelos livianos: tiene buena comprensión semántica, bajo costo computacional y se integra de forma estable con LangChain/HuggingFace.

En la configuración, las decisiones principales fueron:

- `temperature=0.3`: Promueve respuestas controladas y poco creativas, adecuadas para un RAG donde se exige precisión en base a evidencia.
- `repetition_penalty=1.1`: Reduce redundancia, mejorando la claridad de las respuestas generadas.
- `max_new_tokens=512`: Suficiente para responder incluso preguntas complejas sin riesgo de sobreextenderse.
- `timeout` de 120 segundos: Permite ejecutar razonamiento + tool-calls sin abortar por demoras, especialmente útil al interactuar con dos índices vectoriales.

Prompt Engineering

El prompt engineering cumple un rol esencial en el Agentic RAG, ya que regula el comportamiento del LLM, define cuándo es válido generar contenido y determina cómo debe interactuar con las herramientas especializadas del sistema. En un pipeline donde la recuperación y la generación están explícitamente separadas, el prompt debe asegurar que el modelo genere contenido únicamente cuando exista evidencia factual.

Durante el desarrollo, se fue ajustando el diseño de prompts en base a pruebas, llegando al diseño actual.

La estrategia aplicada combina tres tipos de prompts:

- Prompt de Sistema para el nodo principal (chatbot_node): Este es el prompt más importante, ya que estructura el razonamiento del LLM dentro del grafo. Sus objetivos principales son:
 - Definir el rol del agente: El LLM se presenta como un asistente experto que responde exclusivamente en español, con un estilo claro y controlado.
 - Describir las herramientas y cuándo usarlas: Cada tool incluye una explicación precisa. Esta descripción es fundamental para que el LLM pueda actuar como un clasificador semántico y seleccionar la herramienta adecuada.
 - Prohibiciones explícitas para evitar alucinaciones: El prompt establece reglas estrictas como no inventar información factual y no agregar atributos no devueltos por las tools. Estas restricciones son críticas para cumplir con los principios del RAG.
 - Manejo de referencias: El modelo aprende a interpretar expresiones como “eso”, “ahí”, “esa película”, “lo anterior”. Estas referencias se resuelven usando la memoria de corto plazo o el último mensaje generado.
 - Normalización conversacional: El prompt instruye al modelo a convertir automáticamente pronombres como “mi”, “mis”, “yo”, cuando el usuario los usa para referirse a sí mismo, evitando confusiones identitarias.
 - Manejo de respuestas breves: Se establecen reglas para interpretar correctamente mensajes como “sí”, “ok”, “dale”, que suelen depender de la pregunta previa.

- Proveer la última respuesta como contexto: El prompt incluye explícitamente la última salida del agente para anclar la interpretación de referencias contextuales.
- Prompt reducido para *small talk*: Para mensajes triviales (saludos, agradecimientos, expresiones sociales), el agente utiliza un prompt reducido que mantiene el rol general pero omite cualquier instrucción sobre herramientas. Esto evita activaciones innecesarias de tool-calling, mejora la naturalidad y reduce el costo computacional.
- Prompt enriquecido para tool calling automático: Cuando no se detecta small talk, el mensaje se envía al LLM con el system prompt completo, el historial conversacional y las descripciones de las herramientas (mediante `llm.bind_tools(tools)`). Esta combinación habilita el tool-calling nativo de HuggingFace + LangChain. El modelo puede decidir de forma autónoma si invocar una tool, varias tools o ninguna, basándose en la semántica de la consulta.

Estrategias de Prompt Engineering Aplicadas

- Instrucciones explícitas y negativas (para reforzar la precisión factual):
 - “NO inventes información.”
 - “Usa EXCLUSIVAMENTE los datos devueltos por la herramienta.”
- Descripciones acotadas de herramientas: Facilitan que el modelo seleccione la tool correcta sin necesidad de clasificadores externos.
- Control del tono e idioma: Asegura coherencia en todo el diálogo.
- Separación entre prompts de razonamiento y prompts de small talk: Evita activar el grafo completo en situaciones donde no corresponde.

Ejemplos de prompts reales utilizados

Ejemplo 1: System Prompt Principal (fragmento)

“Eres un asistente experto que siempre responde en español con acceso a tres herramientas:

- buscar_peliculas
- buscar_papers
- consultar_memoria

Cuando uses una herramienta, debes basarte EXCLUSIVAMENTE en los datos que devuelva.

No inventes películas, títulos, teorías ni definiciones.

Si una herramienta no devuelve resultados, responde exactamente:

“No encontré información relevante en el dataset.””

Ejemplo 2: Prompt para *small talk*

“Eres un asistente experto que responde en español.

No uses herramientas para este tipo de mensaje.

Da una respuesta breve y natural.”

Conclusión

El diseño de prompts fue determinante para garantizar que el agente combinara correctamente razonamiento, tool-calling y generación basada en evidencia. Al apoyarse en instrucciones claras, restricciones estrictas y descripciones bien acotadas de las herramientas, el sistema logra comportarse de manera controlada, precisa y sin alucinaciones.

Desafíos encontrados y soluciones implementadas

Durante el desarrollo del pipeline Agentic RAG surgieron diversos desafíos que obligaron a iterar sobre el diseño inicial. Las principales decisiones de diseño surgieron de pruebas comparativas, errores de integración y ajustes empíricos para mejorar estabilidad y precisión.

1. Mezcla de idiomas en la generación

Desafío: Al comienzo el agente mezclaba inglés y español en las respuestas, dado que las fuentes de información estaban en inglés y las consultas en español.

Para corregirlo, se probó usar un modelo adicional de embeddings específicamente focalizado en traducción. La estrategia funcionaba, pero con más pruebas se llegó a una solución más simple.

Solución: Se descubrió que el enfoque más estable era mantener los datos del dataset (Letterboxd + papers) completamente en inglés, sin traducirlos y mantener las tools, lógica del grafo, la system prompt y la descripción de herramientas en español, con una instrucción explícita al modelo: “Responde SIEMPRE en español.”

Este diseño eliminó completamente la mezcla de idiomas, evitó pasos adicionales de traducción y mantuvo el flujo simple y estable dentro del grafo.

2. Problemas con el AgentState y la memoria interna

Desafío: La memoria como parte aislada del AgentState no se comportaba de manera completamente consistente: a veces no persistía entre nodos o era sobrescrita accidentalmente.

Solución: Se agregó un array global auxiliar que replica las últimas interacciones, actúa como respaldo para consultas desde tools.

Esto resultó más estable, y permitió al LLM resolver referencias (“eso”, “lo anterior”) sin errores.

3. Nodo separado para *small talk*

Desafío: Al principio se intentó diseñar un nodo dedicado exclusivamente a detectar *small talk* antes de entrar al grafo principal.

Sin embargo, esto complicaba la lógica del grafo y generaba bifurcaciones innecesarias.

Solución: Se integró la detección en el propio chatbot_node mediante la función `es_smalltalk()`. Esto redujo la complejidad y mejoró la legibilidad del grafo. La decisión se determina antes de aplicar tool-calling, sin ramificaciones externas.

4. Top-k demasiado bajo (Top 3) en la búsqueda semántica del dataset de películas

En la herramienta `buscar_peliculas`, la configuración inicial de recuperación usaba `top_k = 3`. Si bien funcionaba para consultas amplias, fallaba cuando el usuario hacía preguntas muy específicas o combinaba múltiples filtros. Muchas veces no aparecían películas que sí existían en el dataset o el modelo terminaba creyendo que no había evidencia.

Solución: Se incrementó el valor a `top_k = 10`, solo en la recuperación semántica de películas, lo cual amplía ligeramente el costo, pero mejora significativamente la cobertura, sin generar ruido excesivo.

Fue un punto de equilibrio óptimo entre precisión y recall.

5. “Consultar memoria” como feature interna (sin tool-calling)

Desafío: Inicialmente, la memoria se resolvía sin una tool explícita: el LLM veía directamente la memoria del estado. Esto funcionaba, pero disminuía el control del razonamiento.

Solución: Convertir la memoria en una herramienta explícita (`consultar_memoria`) con su propio mecanismo de tool-calling.

Esto permitió mayor transparencia, trazabilidad del razonamiento, evitar que el LLM alucine con cosas “que supuestamente recuerda” y habilitar mejores resultados recuperando desde el array global.

Pruebas y resultados obtenidos

Para validar el funcionamiento completo del pipeline agentic RAG, se realizó una serie de pruebas utilizando un conjunto fijo de consultas representativas. Esto permitió evaluar de manera consistente la ejecución del grafo, el comportamiento del agente, la selección de herramientas y el correcto uso de la memoria.

Para analizar las respuestas desde dos perspectivas distintas, se implementaron dos funciones de prueba:

- Ejecución directa: Función que ejecuta el grafo completo y devuelve únicamente la respuesta final del agente. Esta modalidad permitió verificar la calidad de las respuestas sin necesidad de analizar el proceso interno.
- Ejecución con stream paso a paso: Función alternativa que imprime el proceso en tiempo real, mostrando los nodos ejecutados, posibles llamadas a herramientas, resultados intermedios y la generación final. Esta vista detallada permitió depurar decisiones del agente (por ejemplo, cuándo detectaba small talk, cuándo llamaba a cada tool, cómo procesaba la memoria reciente y qué información recuperaba).

El uso combinado de un conjunto fijo de consultas y estas dos funciones permitió observar el comportamiento del agente como caja negra/ caja blanca, facilitando la depuración y ajuste del sistema.

En el notebook entregado se presentan los resultados de las ejecuciones de ambas funciones con el conjunto fijo de consultas. A continuación se hace un breve análisis del motivo de cada consulta y por qué la respuesta obtenida es correcta.

Evidencia

1. Saludo Conversacional (Small-talk)

Prompt: “Hola, ¿cómo estás?”

Respuesta: Hola, estoy bien, gracias por preguntar 😊 ¿Y tú cómo estás?

Objetivo: Prueba de small-talk sin *tool-calling*.

Evaluación: Evaluar que el agente responda conversacionalmente sin activar memoria vectorial o herramientas.

Resultado Esperado: Respuesta correcta porque no usó herramientas y contestó de forma natural.

2. Memoria a Corto Plazo (Ingreso de Dato)

Prompt: “Bien, muchas gracias. Mi color favorito es el azul”

Respuesta: Interesante, el azul es un color muy elegante y tranquilo. ¿Te gustaría que te sugiriera alguna película con un tono azul en su estética o temática? O tal vez te interese saber algo más sobre el azul en el contexto de la cinematografía. 😊

Objetivo: Prueba de memoria a corto plazo.

Evaluación: Se introduce información para que el agente la recuerde luego.

Resultado Esperado: Respuesta correcta porque acepta la información sin activar herramientas.

3. Recuperación Simple (RAG sobre Dataset)

Prompt: “¿Cuáles son 3 películas estrenadas en los años 2000?” **respuesta:**

Aquí tienes 3 películas estrenadas en los años 2000:

1. **Burnt Money** (Argentina, 2000)

- Duración: 125 minutos
- Idioma: Español
- Género principal: Crimen
- Tipo de producción: Independiente

2. **Equilibrium** (EE.UU., 2002)

- Duración: 107 minutos
- Idioma: Inglés
- Género principal: Thriller
- Tipo de producción: Independiente

3. ****Once Upon a Time in Mexico**** (EE.UU., 2003)

- Duración: 102 minutos
- Idioma: Inglés
- Género principal: Acción
- Tipo de producción: Independiente

Objetivo: Prueba de RAG (Generación Aumentada por Recuperación) sobre el dataset.

Evaluación: Evaluar *embeddings* + búsqueda semántica + recuperación real.

Resultado Esperado: Respuesta correcta porque lista películas presentes en el dataset.

4. **Síntesis de Documentos Largos (RAG sobre Papers)**

Prompt: “Qué dice uno de los papers sobre el uso de embeddings en RAG?”

Respuesta: Los papers mencionan que los embeddings son utilizados en el contexto de Retrieval-Augmented Generation (RAG) para representar información de manera vectorial, permitiendo la búsqueda de contenidos relevantes en grandes bases de datos. Específicamente, se destaca que modelos como RoBERTa y otros enfoques de preentrenamiento de embeddings proporcionan representaciones contextuales profundas que se integran en tareas de NLP, como la traducción y la clasificación semántica. En el contexto de RAG, estos embeddings permiten recuperar información relevante de una base de conocimiento antes de generar una respuesta, mejorando así la precisión y coherencia del contenido generado. Además, se menciona que el uso de embeddings multilingües y preentrenados (como en el caso de la arXiv 2001.08210) mejora la eficacia del proceso de recuperación en tareas de traducción y clasificación.

Objetivo: Prueba de RAG sobre documentos largos.

Evaluación: Evaluar *chunking* + *embeddings* + síntesis.

Resultado Esperado: Respuesta correcta porque resume contenido real del *paper* sin inventar.

5. **Búsqueda por Atributo (Metadatos)**

Prompt: “Cuales son algunas películas hechas en USA?”

Respuesta: Algunas películas hechas en Estados Unidos son:

- **Food, Inc.** (2008): Documental de 94 minutos, producción independiente, en inglés.
- **Encanto** (2021): Comedia de 102 minutos, producción independiente, en inglés.
- **Hedda** (2025): Thriller de 107 minutos, producción independiente, en inglés.
- **Piranha** (1978): Ciencia ficción de 94 minutos, producción independiente, en inglés (clásico).
- **Carlito's Way** (1993): Thriller de 144 minutos, producción comercial, en inglés.
- **Saludos Amigos** (1942): Música de 42 minutos, producción independiente, en inglés (clásico).
- **Good Burger** (1997): Comedia de 95 minutos, producción independiente, en inglés.
- **Catopolis** (2022): Animación de 14 minutos, producción independiente, en inglés.
- **The Good, the Bart, and the Loki** (2021): Familia de 6 minutos, producción independiente, en inglés.
- **Soylent Green** (1973): Misterio de 97 minutos, producción independiente, en inglés (clásico).

Estas películas representan una variedad de géneros, décadas y estilos, desde clásicos hasta producciones recientes.

Objetivo: Prueba de recuperación por atributo (país).

Evaluación: Evaluar búsqueda en la base vectorial según metadatos.

Resultado Esperado: Respuesta correcta porque devuelve películas reales del dataset.

6. Combinada: RAG + Reasoning + Memoria

Prompt: "Gracias! Contame primero sobre embeddings y después respondé ¿cuál era mi color favorito?"

Respuesta: Los **embeddings** son representaciones numéricas que permiten convertir datos no numéricos (como palabras, frases o imágenes) en vectores de números, facilitando su procesamiento por modelos de aprendizaje automático. En el contexto de modelos de lenguaje como BERT o Transformer, los embeddings se utilizan para capturar significados contextuales de palabras. Por ejemplo:

- Los **embeddings de palabras** permiten que un modelo entienda que "casa" y "hogar" son conceptos similares.
- Los **embeddings posicionales** añaden información sobre la posición de una palabra en una oración, lo cual es crucial para modelos como los Transformers.
- Los embeddings también se usan en tareas como la clasificación, traducción y reconocimiento de voz.

En resumen, los embeddings son una forma de "traducir" información humana a un formato computacional que los modelos pueden entender y procesar.

Respecto a tu pregunta: **mi color favorito es el azul**. 😊

¿Te gustaría que te recomendara alguna película con un tono azul en su estética o temática?

Objetivo: Prueba combinada: RAG + *reasoning* + memoria a corto plazo.

Evaluación: Evaluar que siga el orden de instrucciones y combine capacidades.

Resultado Esperado: Respuesta correcta porque explica *embeddings* desde un *paper* y recuerda "azul".

7. Consulta Multi-Criterio (Filtros Implícitos)

Prompt: "Cuales son algunas películas hechas en España de terror?"

Respuesta: Aquí tienes algunas películas hechas en España de terror:

1. **The Bar (2017)**

- Duración: 102 minutos
- Género principal: Terror

- Idioma: Español

- Época: Contemporánea

2. **Pieces (1982)**

- Duración: 85 minutos

- Género principal: Terror

- Idioma: Español

- Época: Moderna

3. **Veneciafrenia (2021)**

- Duración: 100 minutos

- Género principal: Terror

- Idioma: Español

- Época: Contemporánea

4. **Horror Express (1972)**

- Duración: 87 minutos

- Género principal: Terror

- Idioma: Inglés

- Época: Época de Oro

5. **Rapture (1979)**

- Duración: 110 minutos

- Género principal: Terror

- Idioma: Español

- Época: Época de Oro

6. **Shrew's Nest (2014)**

- Duración: 91 minutos
- Género principal: Terror
- Idioma: Español
- Época: Contemporánea

7. ****Nightmare City (1980)****

- Duración: 91 minutos
- Género principal: Terror
- Idioma: Italiano
- Época: Época de Oro

8. ****[REC]³ Genesis (2012)****

- Duración: 80 minutos
- Género principal: Terror
- Idioma: Español
- Época: Contemporánea

9. ****Who Can Kill a Child? (1976)****

- Duración: 112 minutos
- Género principal: Terror
- Idioma: Español
- Época: Época de Oro

10. ****Fragile (2005)****

- Duración: 101 minutos

Objetivo: Prueba de consulta multi-criterio (país + género).

Evaluación: Evaluar la coherencia del RAG con filtros implícitos/combinados.

Resultado Esperado: Respuesta correcta porque devuelve películas reales según ambos criterios.

8. Memoria Conversacional Inmediata (Historial)

Prompt: “Me podés repetir la respuesta más reciente que me diste?”

Respuesta:

Aquí tienes algunas películas hechas en España de terror:

1. ****The Bar (2017)****

- Duración: 102 minutos
- Género principal: Terror
- Idioma: Español
- Época: Contemporánea

2. ****Pieces (1982)****

- Duración: 85 minutos
- Género principal: Terror
- Idioma: Español
- Época: Moderna

3. ****Veneciafrenia (2021)****

- Duración: 100 minutos
- Género principal: Terror
- Idioma: Español
- Época: Contemporánea

4. ****Horror Express (1972)****

- Duración: 87 minutos

- Género principal: Terror

- Idioma: Inglés

- Época: Época de Oro

5. ****Rapture (1979)****

- Duración: 110 minutos

- Género principal: Terror

- Idioma: Español

- Época: Época de Oro

6. ****Shrew's Nest (2014)****

- Duración: 91 minutos

- Género principal: Terror

- Idioma: Español

- Época: Contemporánea

7. ****Nightmare City (1980)****

- Duración: 91 minutos

- Género principal: Terror

- Idioma: Italiano

- Época: Época de Oro

8. ****[REC]³ Genesis (2012)****

- Duración: 80 minutos

- Género principal: Terror

- Idioma: Español

- Época: Contemporánea

9. ****Who Can Kill a Child? (1976)****

- Duración: 112 minutos
- Género principal: Terror
- Idioma: Español
- Época: Época de Oro

10. ****Fragile (2005)****

- Duración: 101 minutos

Objetivo: Prueba de memoria conversacional inmediata.

Evaluación: Evaluar el historial del agente.

Resultado Esperado: Respuesta correcta porque repite el mensaje anterior sin usar herramientas.

9. Cita de Fuente (Anti-alucinación)

Prompt: "Haz un resumen sobre la ventaja del uso de embeddings y decime la fuente"

Respuesta: Los embeddings, especialmente en modelos como los transformers, ofrecen una representación densa y contextual de palabras o objetos, lo que permite que los modelos entiendan mejor el significado y la relación entre diferentes elementos. Una de sus principales ventajas es que permiten una escalabilidad significativa en el tamaño del modelo (scale law), ya que al usar embeddings, las operaciones de forward pass se pueden optimizar mediante multiplicaciones matriciales, lo que mejora la eficiencia computacional. Además, en trabajos como el de "Bidirectional Encoder Representations from Transformers" (BERT), los embeddings permiten una comprensión bidireccional del lenguaje, lo que mejora la capacidad del modelo para entender el contexto.

Fuente: arXiv preprint arXiv:1810.04805 (2018) – "Bidirectional Encoder Representations from Transformers" (BERT). Este trabajo demuestra cómo los embeddings contextuales permiten una mejor representación del lenguaje, fundamental para tareas de comprensión y traducción.

Objetivo: Prueba de recuperación de *papers* + cita de fuente.

Evaluación: Evaluar anti-alucinación y síntesis de contenido real.

Resultado Esperado: Respuesta correcta porque resume lo que dice el *paper* y cita el documento (referenciado en los papers cargados).

10. Cierre Conversacional

Prompt: “Gracias!”

Respuesta: De nada, ¿en qué puedo ayudarte ahora? 😊

Objetivo: Prueba de cierre conversacional.

Evaluación: Evaluar *small-talk* sin *tool-calling*.

Resultado Esperado: Respuesta correcta porque responde naturalmente sin activar herramientas.

Interfaz con Gradio

Se implementó una interfaz interactiva utilizando Gradio para facilitar las pruebas del agentic RAG. En lugar de enviar manualmente una secuencia de prompts desde el notebook, la interfaz permitió interactuar con el agente mediante un chat en tiempo real, manteniendo el historial visible y ejecutando el flujo completo del grafo (razonamiento, tool-calling y memoria) de manera natural.

La interfaz se construyó mediante un módulo que vincula la entrada del usuario con el grafo de LangGraph, preservando el AgentState entre turnos. Esto permitió evaluar el comportamiento del sistema en escenarios conversacionales más realistas y detectar rápidamente errores en la detección de intención, el uso de herramientas y el manejo de memoria.

Si bien no era un requerimiento obligatorio, la integración con Gradio resultó muy útil para probar y ajustar el funcionamiento del agente de forma más fluida y cómoda. Esta interacción continua permitió iterar y corregir comportamientos sin necesidad de realizar análisis o comparaciones extensas en el notebook, acelerando significativamente el proceso de desarrollo.

Investigación: posibles mejoras

En un futuro, el sistema podría mejorarse en tres áreas que impactarían positivamente en su funcionamiento.

1. Agentes más profundos (*Deep Agents*)

Una posible mejora es convertir el agente actual en un “*deep agent*”. Esto implica que, en lugar de responder en un único paso, el agente pueda desarmar la pregunta del usuario en varias tareas, planificar los pasos necesarios, consultar varias herramientas en secuencia y verificar si necesita más información antes de responder. Este tipo de arquitectura hace que el agente sea más robusto y capaz de resolver consultas más complejas, acercándose al funcionamiento de los agentes modernos utilizados.

2. Mejor selección de información recuperada (re-ranking avanzado)

Hoy el sistema recupera los documentos más similares y responde directamente. Una mejora concreta sería agregar una etapa donde el modelo vuelva a evaluar esos documentos y elija sólo los más relevantes antes de generar la respuesta. Esto reduce ruido, mejora la precisión y evita que el agente utilice evidencia poco útil. Esta técnica, conocida como “re-ranking”, es estándar en los RAG más avanzados.

3. Memoria de largo plazo

El sistema actual solo recuerda los últimos mensajes de la conversación. Una mejora importante sería agregar una memoria de largo plazo, capaz de almacenar preferencias, información importante o patrones del usuario entre sesiones. Esto permitiría que el agente aprenda del usuario y mantenga continuidad real, y no solo dentro de una única interacción.

Fuentes consultadas

[1] LangChain, “LangGraph Overview,” LangChain Documentation. Accessed: Nov. 15, 2025. Available: <https://docs.langchain.com/oss/javascript/langgraph/overview>

[2] T. Saeed, “Chunking and Embedding Strategies in RAG: A Guide to Optimizing Retrieval Augmented Generation,” Medium, Oct. 2023. Accessed: Oct. 30, 2025. Available: https://medium.com/@tahir.saeed_46137/chunking-and-embedding-strategies-in-rag-a-guide-to-optimizing-retrieval-augmented-generation-7c95432423b1

[3] Microsoft, “Prompt engineering for Azure OpenAI,” Microsoft Learn. Accessed: Nov. 24, 2025. Available: <https://learn.microsoft.com/en-us/azure/ai-foundry/openai/concepts/prompt-engineering>

[4] LangChain, “Planning and Multi-step Agents,” LangChain Documentation. Accessed: Nov. 25, 2025. Available: <https://python.langchain.com/docs/modules/agents/>

[5] Microsoft, “LLM-as-a-Judge: When LLMs Rerank Documents Better than Retrievers,” arXiv preprint, 2024. Accessed: Nov. 25, 2025. Available: <https://arxiv.org/abs/2403.00109>

[6] OpenAI, “Memory in GPT Agents,” OpenAI Documentation. Accessed: Nov. 25, 2025. Available: <https://openai.com/index/memory/>